



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Centro de
e-Learning
Secretaría de Cultura y Extensión Universitaria

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Buenos Aires

INTELIGENCIA ARTIFICIAL APLICADA A ORGANIZACIONES

Trabajo de fin de Ciclo

ENTREGA FINAL DE PROYECTO

INTELIGENCIA ARTIFICIAL APLICADA A ORGANIZACIONES

Universidad Tecnológica Nacional · Facultad Regional Buenos Aires

Alumnos: Daniel Galindez y Paulo Ramirez

Links obligatorios

Recurso	URL / estado verificable
Repositorio GitHub	https://github.com/dan-kun/project-constructor-ia
Aplicación web en producción	https://project-constructor-ia.onrender.com <i>Alojada en plan gratuito: si estuvo inactiva, la primera carga puede demorar hasta ~50 s mientras la instancia se reactiva. Las siguientes son inmediatas.</i>
Documentación técnica	https://dan-kun.github.io/project-constructor-ia/ (sitio publicado); también en el repositorio: README.md y carpeta docs/.
Video de demostración (3 min)	https://drive.google.com/file/d/1t78LkvLC-cXF-BkJCP8yGZjwaf1A0rbvU/view <i>Acceso público: no requiere iniciar sesión ni solicitar permiso.</i>

Fecha de elaboración: agosto de 2026.

1. Presentación del equipo y del proyecto

1.1 Integrantes y roles

Daniel Galíndez — Arquitectura y núcleo del sistema. Diseño de la arquitectura hexagonal y del puerto LLMProvider; modelo de dominio y contratos tipados; orquestador y máquina de estados con sus ciclos acotados; los seis agentes y las plantillas por stack; capa de verificación (sintaxis, build en Docker y smoke tests); suite de pruebas e integración continua; documentación técnica e integración de las ramas de trabajo.

Paulo Ramírez — Interfaz web, experiencia de uso y despliegue. Adaptador web sobre el mismo orquestador (frontend y sesiones con Server-Sent Events); descarga del proyecto generado y de la transcripción de la sesión; recuperación de sesiones interrumpidas y límite de tasa; ajuste de los prompts de los agentes; diagramas de la entrega; publicación de la aplicación en Render.

Ambos integrantes condujeron el desarrollo con asistentes de IA en co-work y aplicaron revisión cruzada entre modelos sobre el trabajo del otro, según se detalla en la Sección 7. La división de responsabilidades refleja la separación que propone la propia arquitectura del sistema: el núcleo determinístico por un lado, los adaptadores de entrada/salida y la publicación por el otro.

1.2 Nombre y problema

Project Constructor IA (PCIA) es una aplicación para crear de forma asistida, auditada y verificable la estructura inicial de proyectos de software. El problema de partida es que iniciar un proyecto exige decisiones repetitivas pero de alto impacto —stack, arquitectura, secretos, despliegue, CI/CD y pruebas— y suele depender de perfiles senior. PCIA convierte ese proceso en una entrevista guiada, revisa incoherencias antes de escribir archivos, genera un scaffold a partir de plantillas controladas y verifica lo construido con validaciones reales.

La solución implementada no promete generar cualquier tecnología sin control: hoy soporta plantillas FastAPI, NestJS y módulos Odoo. Si se pide un stack sin plantilla, escala la situación al usuario en lugar de improvisar. El resultado esperado incluye el árbol de archivos, README, ADR, reporte de verificación y registro de memoria del proyecto.

1.3 Público objetivo

El público objetivo son desarrolladores, líderes técnicos y consultoras/fábricas de software que inician proyectos recurrentemente. El usuario entiende conceptos básicos de desarrollo, pero necesita apoyo para transformar requisitos en una base consistente y segura. La interfaz está en español rioplatense y expone el avance como entrevista, semáforo de coherencia y resultados de verificación; no está diseñada como herramienta no-code para personas sin conocimiento técnico.

2. Arquitectura técnica

2.1 Arquitectura general

La arquitectura es hexagonal. El núcleo contiene el dominio tipado y un orquestador determinístico que funciona como máquina de estados; los modelos de IA no deciden el flujo global. Las interfaces CLI y web son adaptadores intercambiables. Toda llamada a un modelo cruza el puerto LLMProvider, por lo que pueden usarse Anthropic, una API compatible con OpenAI, Ollama/llama.cpp/LM Studio o la suscripción de Claude en modo headless.

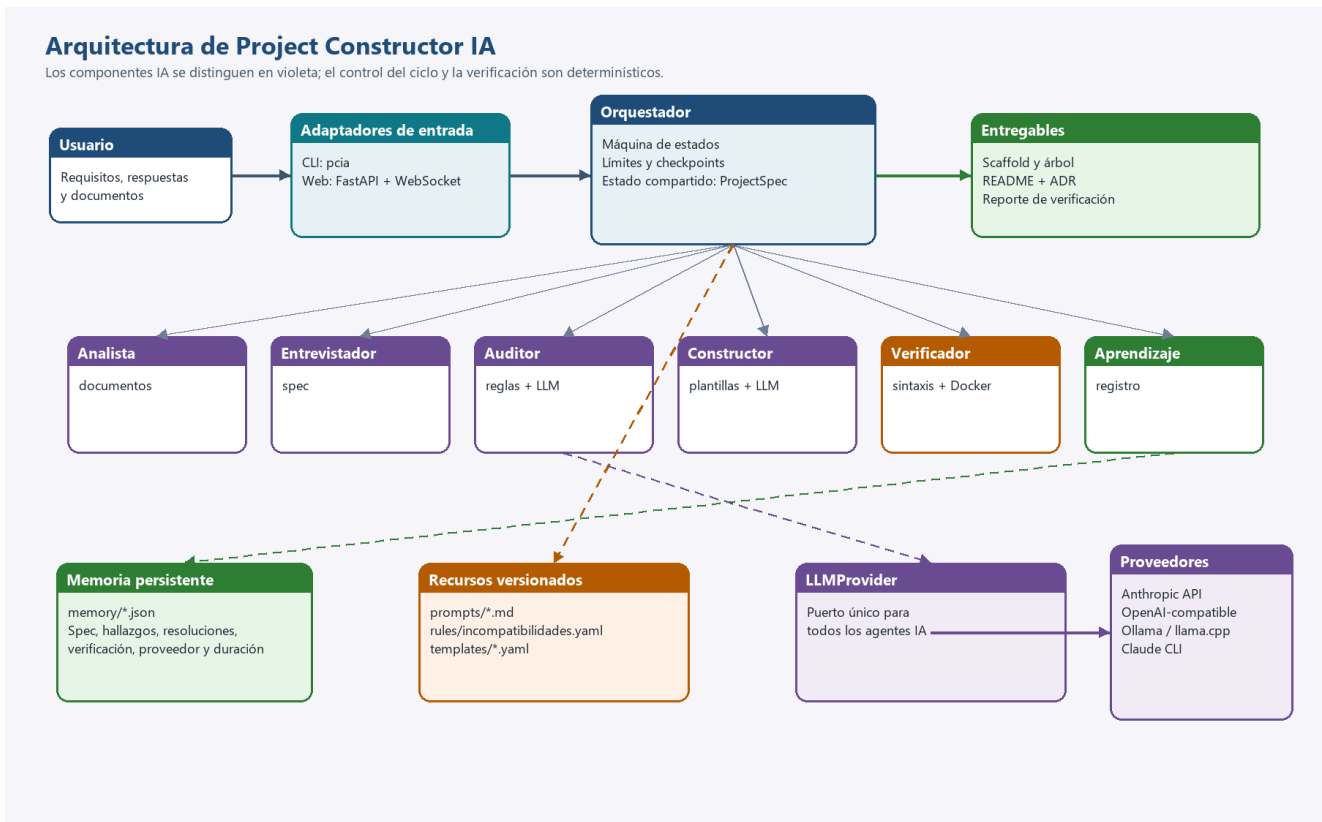


Figura 1. Arquitectura general: componentes IA, lógica tradicional, memoria y entregables.

Flujo de datos: el usuario aporta respuestas y, opcionalmente, documentos de requisitos; el Analista propone datos con evidencia; el Entrevistador completa ProjectSpec; el Auditor emite hallazgos; el Constructor renderiza una plantilla; el Verificador analiza sintaxis, referencias del README, build, smoke test y linter cuando corresponde; finalmente se persiste un RegistroProyecto JSON. Los agentes no se invocan entre sí: colaboran mediante ProjectSpec y resultados tipados administrados por el orquestador.

2.2 Ciclo de agentes

Fase / agente	Decisión o responsabilidad	Salida y siguiente paso
Analista (opcional)	Extrae propuestas desde documentación del cliente; exige evidencia textual.	Propuestas confirmables; pasa a entrevista.



Entrevistador	Pregunta de manera adaptativa y completa los campos de ProjectSpec.	Especificación completa y confirmada; pasa a auditoría.
Auditor	Cruza reglas determinísticas y una revisión LLM. Rojo bloquea; amarillo se corrige o queda como riesgo asumido.	Vuelve a entrevista si hay ajuste; continúa si queda en verde.
Constructor	Selecciona plantilla soportada, renderiza archivos, README y ADR; evita escribir hasta tener el resultado completo.	Scaffold generado; pasa a verificación.
Verificador	Valida sintaxis y documentación; ejecuta build/smoke Docker y lint según la plantilla. Los reintentos están acotados.	Aprobado, aprobado con advertencias o inconcluso; pasa a entrega.
Aprendizaje	Guarda especificación, resoluciones, chequeos, proveedor y duración.	Registro JSON disponible para futuras preferencias.

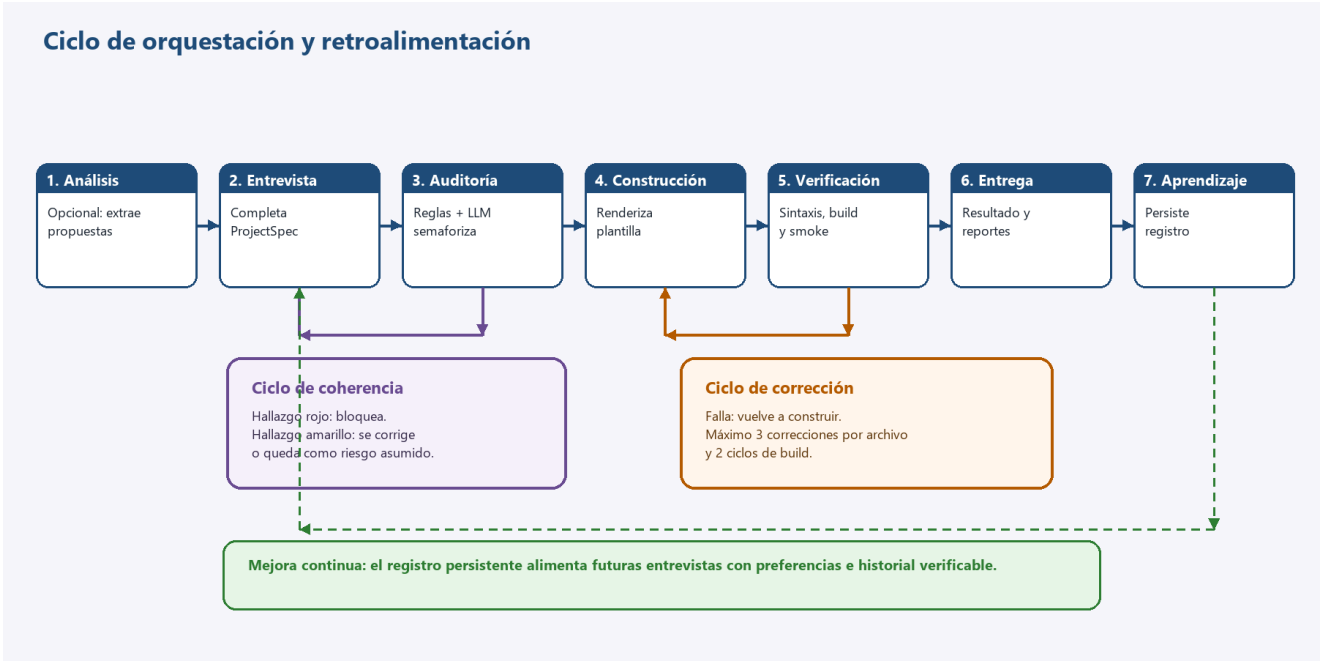


Figura 2. Ciclo de orquestación con retroalimentación de coherencia, corrección y aprendizaje.

2.3 UML: modelo de dominio resumido

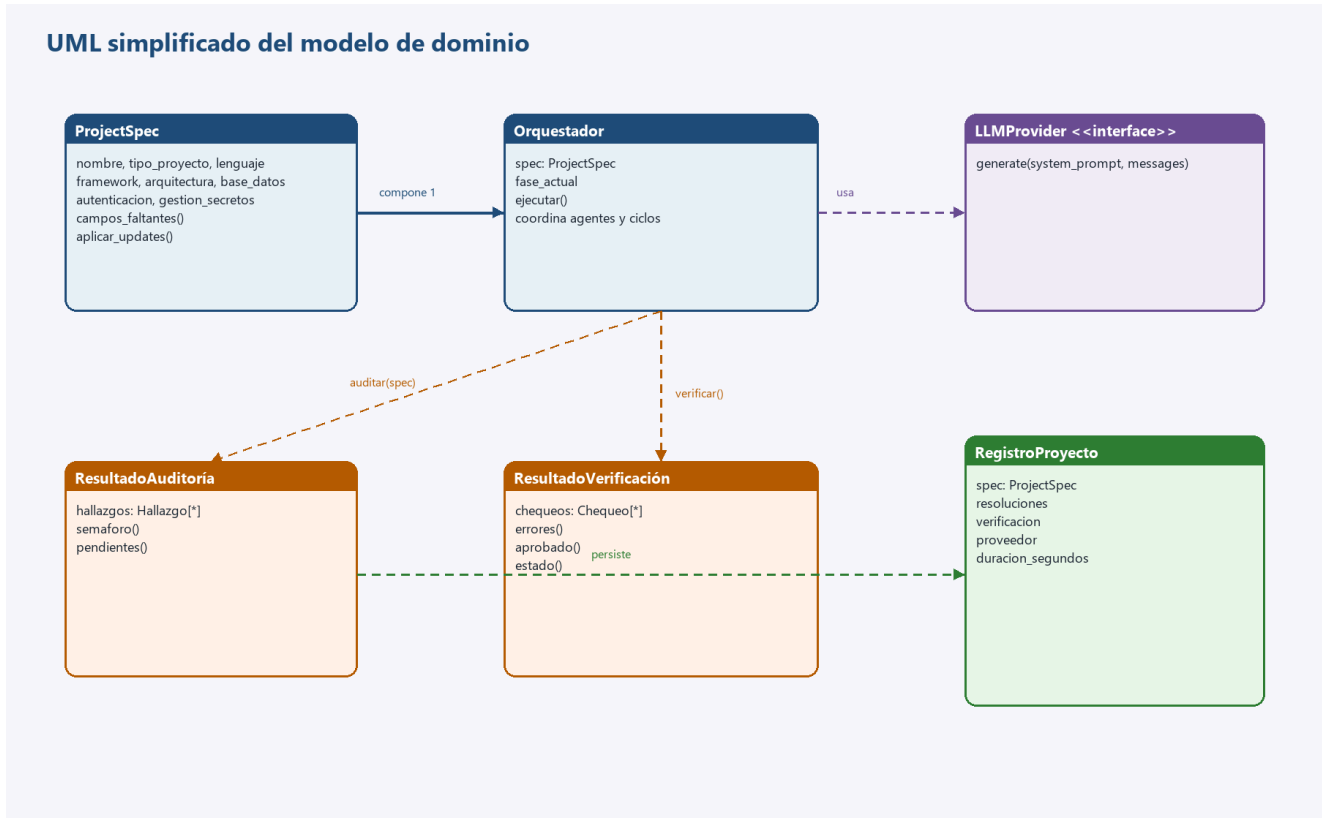


Figura 3. UML simplificado: estado compartido, orquestador, contratos y persistencia.

ProjectSpec es la pizarra única compartida. Los contratos se implementan con Pydantic y prohíben campos extra, de modo que una salida inventada por el modelo no ingresa silenciosamente al ciclo. La memoria usa un JSON por proyecto porque el volumen actual es bajo y el registro puede inspeccionarse sin una base de datos ni migraciones.

3. Stack tecnológico

Componente	Tecnología / herramienta	Por qué se eligió
Frontend	HTML, CSS y JavaScript nativos	Una única vista conversacional actualizada por Server-Sent Events (SSE); evita un toolchain de frontend sin aportar valor al alcance actual.
Backend	Python ≥ 3.10; FastAPI y Uvicorn (extra web)	Python facilita la integración con IA; FastAPI aporta ASGI y Server-Sent Events para sesiones largas, que atraviesan proxies y funcionan en hosting gratuito.
Dominio y validación	Pydantic 2	Contratos tipados para ProjectSpec, hallazgos y salidas LLM; extra='forbid' evita campos inesperados.
Memoria persistente	Archivos JSON en memory/	Decenas de registros, auditables a simple vista y sin dependencia ni migración. Reevaluable si crece el volumen.

Modelo de IA	Anthropic API; OpenAI-compatible; Claude CLI; Ollama/llama.cpp/LM Studio	Se intercambian mediante LLMPProvider; habilitan costo, calidad o privacidad según contexto.
Orquestación	Código propio: máquina de estados	El flujo y sus límites son explícitos y auditables; se evitó acoplamiento a un framework de agentes.
Reglas y plantillas	YAML y Markdown versionados	Las incompatibilidades, prompts y scaffolds se revisan como datos y se modifican sin alterar el núcleo.
Verificación	Parseres de sintaxis, Docker, smoke tests y ruff	Verifica ejecución real cuando Docker está disponible; un build omitido deja el resultado inconcluso.
CI	GitHub Actions + pytest	La suite se ejecuta en Python 3.10 y 3.12 en cada push.
Despliegue	Render (configuración render.yaml)	Existe configuración para Uvicorn y health check; falta evidenciar una URL pública activa.

4. Evidencia de funcionamiento

Evidencia disponible en el repositorio: el README documenta la interfaz web y la CLI; los diagramas y la secuencia de una corrida real están en docs/ARQUITECTURA.md; docs/LLM-LOCAL.md documenta una corrida end-to-end contra Qwen3-30B-A3B local con FastAPI en verde y una corrida NestJS que detectó un defecto real de plantilla. El historial Git muestra evolución incremental y merge de la integración web, no un único commit final.

Evidencia	Qué demuestra	Ubicación
Interfaz web	Chat, panel de estado, semáforo, archivos y verificaciones visibles.	src/pcia/web/; README.md
CLI	El mismo núcleo puede ejecutarse en consola y acepta documentos con --docs.	src/pcia/cli.py; README.md
Corrida local	Auditoría, construcción, Docker build y smoke test de FastAPI; defecto detectado en NestJS.	docs/LLM-LOCAL.md y docs/IA-COWORK.md
Pruebas automatizadas	277 pruebas automatizadas en verde. La suite es hermética: usa un proveedor falso, sin red ni Docker.	tests/ · Ejecutadas también por CI en Python 3.10 y 3.12 sobre un entorno limpio en cada push (badge en el README).



Historial	Commits reales y trabajo incremental; remoto GitHub configurado.	https://github.com/dan-kun/project-construct-or-ia
Log de sesión real	Recorrido completo: entrevista, hallazgo amarillo asumido y propagado, scaffold con PostgreSQL materializado y verificación en verde (syntax, build, smoke y linter).	Anexo A de este informe · descargable desde la app como conversacion.txt

4.1 Capturas requeridas para anexar antes de enviar

Las tres capturas se tomaron de la aplicación en funcionamiento y se incorporan a continuación. Muestran una corrida real: la elección del proveedor, la entrevista con el panel de estado y un hallazgo del semáforo, y la salida con el árbol generado y el reporte de verificación. Complementan —no reemplazan— la evidencia ejecutable del repositorio: el registro de cada sesión puede descargarse desde la propia interfaz como conversación en .txt junto al proyecto en .zip.

Captura	Escena requerida	Estado
Figura A	Home: configuración del proveedor y comienzo de la entrevista.	INCORPORADO
Figura B	Flujo principal: preguntas, ProjectSpec lateral y un hallazgo del semáforo.	INCORPORADO
Figura C	Salida: árbol generado y reporte de verificación/build/smoke.	INCORPORADO

Project Constructor IA

Entrevista, auditoría y construcción de proyectos, con criterio experto verificable.

1. Elegí el proveedor de IA

El sistema es agnóstico del modelo: este formulario simplemente arma el mismo config.yaml que usa la CLI, pero con tus datos, para esta sesión. Nada se guarda en el servidor. Para no retippear esto cada vez, el navegador lo recuerda localmente (localStorage, solo en este equipo) — [clicar lo guardado](#)

☐ Anthropic API (Claude)

☒ Compatible OpenAI (Ollama local, Groq, OpenRouter...)

Base URL

https://openrouter.ai/api/v1

Usar OpenRouter

Modelo

qwen2.5:14b (Ollama local, default del proyecto)

Cargar disponibles

API key (opcional en Ollama local; requerida en Groq/OpenRouter/etc.)

Documentación del cliente (opcional — .md/.txt, hasta 5 archivos)

Choose Files

No file chosen

El Analista de Documentos la lee antes de empezar y propone valores con la cita que los respalda; vos confirmás cada propuesta durante la entrevista, nunca se aplican solas.

Iniciar entrevista

Figura A. Home: configuración del proveedor y comienzo de la entrevista.

● Seguimos esperando al agente (puede tardar más con un modelo local, o si está corriendo un build de Docker)...

Corregir mi última respuesta

1. **Nombre** del proyecto y una **breve descripción** de qué hace o qué problema resuelve.
2. **Tipo de proyecto** (ej. API REST, aplicación web full-stack, CLI, librería, microservicio, etc.) y su **alcance** (prototipo/MVP, producto en producción, herramienta interna, prueba de concepto).

Con eso ya tengo la base para seguir afinando el stack técnico.

el proyecto se llamara autopartes, es un monolito para gestionar y administrar contratos publicos vehiculares, quiero primero implementar el mvp

Perfecto, registrado:
- **nombre**: autopartes
- **descripcion**: monólito para gestionar y administrar contratos públicos vehiculares
- **alcance**: MVP
- **tipo_proyecto**: lo marco como "aplicación web monólito" (sistema de gestión), ¿te parece bien?

Ahora el stack técnico. Para un monólito de gestión con MVP, opciones habituales:
- **Python + FastAPI** (rápido, tipado, buena ecosistema admin)
- **Python + Django** (baterías incluidas: admin, auth, ORM, ideal para MVP)
- **Node.js + NestJS** (estructura modular, TypeScript nativo)
- **Go + Gin/Fiber** (performance, simple deployment)

Escribí tu respuesta...

Enviar

Enter para enviar · Shift+Enter para salto de línea

ESTADO DEL PROYECTO

FASE DEL CICLO

Análisis
Entrevista
Auditoría
Construcción

Verificación
Entrega
Aprendizaje

ESPECIFICACIÓN

nombre	autopartes
descripcion	monolito para gestionar y administrar contratos publicos vehiculares
tipo_proyecto	aplicación web monólito
lenguaje	Python
framework	FastAPI
arquitectura	capas limpias (router → service → repository → models)
base_datos	PostgreSQL
autenticacion	JWT stateless
gestion_secretos	Variables de entorno (.env)
infraestructura	Docker Compose
ci_cd	GitHub Actions
alcance	MVP
riesgos asumidos	5

AUDITORÍA DE COHERENCIA

● **amarillo**

[jwt-sin-refresh-tokens] 11m
JWT stateless sin estrategia de refresh tokens: access tokens cortos (15-30 min) obligan a re-login constante: access tokens largos (días) dejan ventanas de exposición amplias si el token se filtra. Para un MVP de contratos públicos donde los usuarios trabajan sesiones extensas, esta implementación es riesgosa.

Figura B. Flujo principal: preguntas, ProjectSpec lateral y un hallazgo del semáforo.

● Entrevista y construcción finalizadas

Corregir mi última respuesta

Resultado de la verificación profunda:
☒ [docker-build] ok
☒ [smoke-import-app] ok
☒ [lint-ruff] ok

Estado final de la entrega: ☒ aprobado

Especificación y registro del proyecto guardados en
C:\Users\paulo\AppData\Local\Temp\pcia-web-sesiones\616c93639d1d4949b23406ad46698a5e\memory\autopartes-20260817-113332.json
(duración total: 1213s)

Memoria actualizada: 1 proyecto(s) registrado(s).
Preferencias detectadas para precargar la próxima entrevista:
- lenguaje: python (en 1 de 1 proyectos)
- framework: fastapi (en 1 de 1 proyectos)
- arquitectura: monolito-modular (en 1 de 1 proyectos)
- base_datos: postgresql (en 1 de 1 proyectos)
- autenticacion: jwt (en 1 de 1 proyectos)
- gestion_secretos: env (en 1 de 1 proyectos)
- infraestructura: docker-compose (en 1 de 1 proyectos)
- ci_cd: github-actions (en 1 de 1 proyectos)

Escribí tu respuesta...

Enviar

Enter para enviar · Shift+Enter para salto de línea

Descargar conversación (txt)

Descargar proyecto generado (.zip)

gestion_secretos .env

infraestructura docker-compose

ci_cd github-actions

alcance CRUDs básicos para la gestión inicial

AUDITORÍA DE COHERENCIA

● **verde**

[autenticacion-jwt-sin-secretos-cifrados] 11m
La autenticación JWT utiliza secretos almacenados en .env, lo cual es inseguro para despliegues que no sean locales.
! Considerar el uso de un secreto cifrado (por ejemplo, Vault o Secret Manager) para la autenticación JWT si se planea desplegar fuera del entorno local.

SCAFFOLD GENERADO

plantilla	fastapi
archivos	15

```
.env.example
.github/workflows/ci.yml
.gitignore
Dockerfile
README.md
compose.yaml
docs/adr/ADR-001-decisiones-iniciales.md
pyproject.toml
src/autopartes/__init__.py
src/autopartes/config.py
src/autopartes/db.py
src/autopartes/main.py
tests/conftest.py
tests/test_config.py
tests/test_health.py
```

VERIFICACIÓN

☒ aprobado

Figura C. Salida: árbol generado y reporte de verificación/build/smoke.

5. Evaluación UX/UI

La autoevaluación se basa en la implementación web y en la revisión documentada en docs/UX.md. El objetivo es una herramienta para perfiles técnicos, por lo que se conserva vocabulario del oficio, pero se evita que la persona tenga que recordar información previamente dada: el estado, las decisiones y las verificaciones se presentan en el panel lateral.

5.1 Heurísticas de Nielsen

Heurística	Cumple	Evidencia / observación
Visibilidad del estado	Sí	La fase, el semáforo, el árbol y los chequeos se actualizan; hay indicador de actividad.
Coincidencia con el mundo real	Sí	Lenguaje en español, metáfora de semáforo y términos conocidos: build, linter, CI, secreto.
Control y libertad	Parcial	Se puede ajustar la spec, resolver hallazgos y reiniciar con checkpoint. Recargar la web aún pierde la sesión y no existe deshacer granular.
Consistencia y estándares	Sí	Colores y estados consistentes; las opciones fijas se muestran como botones.
Prevención de errores	Sí	Hallazgos rojos bloquean; confirmación antes de auditar; destinos no deben sobrescribir proyectos; confirmación para Dockerfile reescrito.
Reconocimiento antes que recuerdo	Sí	ProjectSpec y hallazgos siempre visibles; el usuario no debe recordar respuestas anteriores.
Flexibilidad y eficiencia	Sí	Memoria y carga de documentos aceleran usuarios recurrentes.
Diseño estético y minimalista	Sí	Dos columnas, paleta sobria y bloques que aparecen cuando hay información; persiste duplicación parcial de texto y panel.
Reconocer y recuperar de errores	Parcial	Chequeos incluyen detalle y diagnóstico. Las excepciones no controladas aún pueden verse crudas y sin acción siguiente.
Ayuda y documentación	No en la UI	README y docs son completos, pero falta onboarding, ayuda contextual y explicación de estados dentro de la interfaz.

5.2 Evaluación orientada al público objetivo

El diseño es adecuado para desarrolladores y líderes técnicos: las decisiones se expresan en conceptos que ya usan, el semáforo traduce la prioridad y las verificaciones dan evidencia concreta. Para un público no técnico sería insuficiente porque solicita

decisiones de arquitectura y rutas de destino; sería necesario un modo guiado con explicaciones y defaults de negocio.

El lenguaje es comprensible para el público definido y evita jerga interna del sistema. Sí existe feedback informal de un usuario real documentado en UX.md: reportó que, ante un error o una respuesta incorrecta, debía volver a empezar. A partir de ello se agregaron checkpoints, la opción de corregir reutilizando la especificación y el comando CLI `--resume`. Pendientes identificados: recuperación tras recarga, accesibilidad ARIA, selector de destino, ayuda contextual y mensajes de error accionables.

6. Evaluación de ciberseguridad

La seguridad se analizó para tres superficies: PCIA como herramienta que procesa documentación y ejecuta builds, los scaffolds que genera y la aplicación web publicada. La tabla resume medidas implementadas y el riesgo residual declarado en docs/SEGURIDAD.md. Varias medidas están verificadas sobre la instancia publicada y no solo en pruebas: la defensa contra SSRF rechaza efectivamente el endpoint de metadatos de la nube (169.254.169.254) en producción, y la memoria de proyectos se aísla por sesión para que la especificación de un visitante no se persista junto a las demás ni precargue la entrevista de otro.

Riesgo identificado	Tipo	Medida implementada / decisión
Inyección de prompt desde documentación del cliente	Prompt injection	Contratos estrictos; el Analista solo propone campos permitidos con evidencia; confirmación humana; auditoría determinística posterior y delimitación explícita del texto no confiable.
Escritura fuera del scaffold durante correcciones LLM	Ejecución / integridad	El corrector valida que los archivos pertenezcan a la lista permitida y rechaza rutas fuera de ella antes de escribir.
Build de código generado	Ejecución / aislamiento	Docker se usa para build y smoke test; se aplica timeout y limpieza. Riesgo residual: Docker no sustituye una sandbox completa ante un atacante local privilegiado.
Exposición de API keys	Secretos	Claves por variables de entorno/config.local.yaml ignorado por Git; la UI no persiste credenciales de proveedor en memoria compartida.
SSRF desde la configuración de proveedor web	Acceso de red	Validación de URLs externas y política para destinos internos; pruebas cubren URLs no permitidas.
Datos de usuarios y memoria	Privacidad	Memoria aislada por sesión por defecto; solo se persiste el registro del proyecto. Memoria compartida es opt-in.
Secretos generados en scaffolds	OWASP / secretos	Regla roja bloqueante para secretos hardcodeados. Plantilla FastAPI exige SECRET_KEY y usa variables de entorno; contenedores corren sin root.
Abuso de sesiones web	Disponibilidad	Límite de tasa, validación de documentos (tipo, cantidad y tamaño) y limpieza de sesiones expiradas.

Decisión explícita: no se implementó autenticación de usuarios de aplicación como producto multiusuario. La instancia web actual se concibe como herramienta de equipo o demostración y su uso público debe acompañarse de control de acceso del hosting/proxy. Para producción multiusuario quedan pendientes autenticación, persistencia robusta de sesiones, aislamiento reforzado de builds y observabilidad.

7. IAs utilizadas en el co-work de desarrollo

Herramienta IA	Uso	Aporte y límites observados
Claude Code	Asistente principal: implementación por módulos, tests, prompts, documentación y diagnóstico.	Aceleró el andamiaje. Introdujo o no detectó problemas de contexto, como npm ci sin lockfile; se corrigieron mediante ejecución real.
Codex 5.6	Revisión cruzada posterior al feedback de medio ciclo.	Detectó problemas reales, como documentación plausible pero falsa; algunas recomendaciones requerían re-priorización por alcance y fecha.
Qwen3-30B-A3B local (llama.cpp)	Ejecución real del producto como proveedor local.	Permitió verificar privacidad y degradación con gracia; produjo un falso positivo del Auditor sobre variables de entorno.
Modelos de Anthropic / compatibles OpenAI	Proveedores soportados por el puerto LLMPProvider.	Se desacoplaron de la lógica del dominio para cambiar por configuración y no por reescritura.

El co-work con IA redujo sustancialmente el tiempo de desarrollo en el andamiaje repetitivo: módulos, pruebas, adaptadores, prompts y documentación. Sin embargo, la IA no reemplazó el juicio de producto ni la validación. Falló al aplicar una práctica general (npm ci) fuera de su contexto, al generar documentación técnicamente plausible pero falsa y al confundir una práctica correcta de secretos con un riesgo. Por eso el equipo incorporó contratos Pydantic, pruebas herméticas, builds y smoke tests reales, revisión cruzada entre modelos y la decisión humana como última instancia.

PARTE 2 — IA local en el proyecto

1. Papel de un LLM/SLM local

Un LLM local ya puede ocupar el mismo lugar que una API externa porque todas las llamadas pasan por LLMPProvider. Se probó el sistema contra un servidor llama.cpp en <http://localhost:8088/v1> con Qwen3-30B-A3B (GGUF), sin modificar agentes ni orquestador. El modelo participa en Entrevistador, Analista, Auditor, Constructor y corrector; Aprendizaje es determinístico. La evolución recomendable es híbrida: un modelo local para entrevista y documentación —tareas frecuentes y de menor riesgo— y un modelo más capaz para el juicio del Auditor y diagnósticos de build cuando el contexto lo requiera.

2. Aporte al usuario de la aplicación

El aporte principal es confidencialidad. PCIA puede leer requisitos, infraestructura y documentación de clientes; con un modelo local esos datos no salen de la organización, lo que permite usar el Analista en proyectos bajo NDA. También elimina el costo marginal por token de un flujo que incluye varias preguntas y reintentos defensivos. El usuario puede mantener el ciclo aun sin Internet, sin límites de tasa ni cambios de precio de un proveedor. La contrapartida es que la experiencia depende del hardware disponible y puede ser más lenta.

3. Aporte profesional al equipo

Trabajar con un modelo local permitió probar la arquitectura con una capacidad menos predecible que la de una API grande. El falso positivo del Auditor mostró que las garantías reales provenían de las reglas determinísticas y de la confirmación humana, no del modelo. También volvió visible que la ventana de contexto condiciona el producto: el Analista trunca documentación y el corrector excluye archivos irrelevantes para caber en el contexto. En el trabajo diario, disponer de este modo offline habilita analizar documentación o logs sensibles y medir duración/proveedor sin depender de una cuenta externa.

4. Limitaciones frente a una API en la nube

La principal limitación concreta observada es la calidad del juicio técnico: el modelo local marcó como riesgoso el uso de variables de entorno, una práctica correcta. Su ventana de 8192 tokens obliga a truncar documentos extensos y contexto de correcciones; las APIs grandes suelen ofrecer más. El rendimiento depende de RAM/VRAM y de la cuantización: no toda estación de trabajo puede servir un modelo de este tamaño con buena latencia. Finalmente, el costo no desaparece: se traslada al mantenimiento del servidor, selección de modelo, actualizaciones y diagnóstico. Por eso la elección correcta es por tarea y sensibilidad de datos, no una sustitución absoluta nube versus local.

Demostración opcional reproducible

pcia-web

por navegador: el proveedor se elige ahí

Para sumar la evidencia opcional, anexar una captura de esta terminal con la respuesta. No se incorpora una captura artificial en este documento; la configuración y las corridas reales están documentadas en docs/LLM-LOCAL.md.

Conclusión y estado de entrega

Project Constructor IA evolucionó desde el diseño conceptual a una aplicación con núcleo ejecutable: orquestador de estados, agentes especializados, memoria persistente, interfaz web/CLI, plantillas soportadas y verificación de artefactos. Su decisión más relevante es separar el juicio probabilístico del control determinístico: el modelo propone y redacta,



mientras que contratos, reglas, límites y verificaciones protegen el ciclo. Esta separación también permite ejecutar con proveedores de nube o locales sin reescribir el dominio.

Anexo A — Registro de una sesión real de uso

Transcripción completa de una sesión ejecutada con la interfaz web sobre un modelo local (llama.cpp / Qwen3-30B-A3B), descargada desde la propia aplicación con el botón «Descargar conversación (.txt)». Duración total: 1288 s (21 min), coherente con la latencia de un modelo local en hardware de escritorio.

La sesión evidencia, en un único recorrido:

- Entrevista adaptativa hasta completar la especificación, con confirmación explícita antes de auditar.
- Auditoría con hallazgo amarillo y corrección propuesta; el usuario asume el riesgo y queda documentado.
- Propagación del riesgo asumido: la entrega se reporta como «aprobado con advertencias», no como aprobado.
- Materialización condicional de la especificación: al elegir PostgreSQL, el scaffold incluye compose.yaml y el módulo de conexión db.py.
- Verificación en tres capas, toda en verde: sintaxis (11 archivos), build de Docker, smoke test dentro de la imagen y linter.
- Persistencia en memoria y detección de preferencias para precargar la próxima entrevista.

```
¡Bienvenido! Para comenzar, ¿cuál es el nombre de su proyecto y una breve descripción de su propósito?
```

```
> transformar-data
```

```
Perfecto. ¿Podría describir brevemente el propósito de su proyecto?
```

```
> quiero tomar documentos, conexiones a distintas BD, traer info y pasarla por un depurador que convierta y homologue toda la data a una centralización que luego dispondra esta info via API
```

```
Entendido. ¿Cuál es el tipo de proyecto? Por ejemplo, herramienta de integración de datos, sistema ETL, o algo más específico?
```

```
> me parece que un poco de ambos que has mencionado
```

```
Entiendo. ¿Confirmaría que el tipo de proyecto sería una combinación de 'herramienta de integración de datos' y 'sistema ETL'? O ¿prefiere definir un tipo específico diferente?
```

```
> si, vamos con una combinacion
```

```
Perfecto. ¿Cuál es el lenguaje de programación y el framework que prefiere usar para este proyecto? Por ejemplo, Python con Django, Node.js con Express, o cualquier otro que tenga en mente.
```

```
> no tengo uno definido, cual sería el más eficaz, optimo y rapido para este tipo de proyecto?
```



Para un proyecto de integración de datos y ETL, recomendaría Python con FastAPI (para la API) y bibliotecas como Pandas para procesamiento. Es rápido de desarrollar y tiene excelentes herramientas para transformación de datos. ¿Le gustaría explorar esta opción o tiene preferencias específicas?

> me gusta tu recomendación

Excelente elección. Para continuar, ¿cuál es el alcance de su proyecto? Por ejemplo, pequeño (prototipo o funcionalidad específica), mediano (múltiples fuentes de datos y funcionalidades robustas), o grande (escalable y con múltiples módulos)?

> mediano

Entendido. Para un proyecto de alcance mediano, ¿qué arquitectura propone? Por ejemplo, monolítica (sencilla y rápida), en capas (para separación de responsabilidades) o alguna otra. También, ¿qué base de datos utilizaría para la centralización de datos? (ej: PostgreSQL, MongoDB, o alguna específica)

> necesito que sea en capas y usare postgresQL

Perfecto. Para seguridad, ¿qué tipo de autenticación necesitaría? (ej: OAuth2, JWT, o ninguna si es interno). También, ¿cómo gestionaría secretos (clave API, credenciales)? (ej: variables de entorno, AWS Secrets Manager, o ninguna si es público). Finalmente, ¿qué infraestructura usaría? (ej: nube (AWS/GCP), servidor propio, o local). Y ¿qué sistema CI/CD preferiría? (ej: GitHub Actions, Jenkins, o ninguno)

> OAuth2, variables de entorno, servidor propio y GitHub Actions o posiblemente gitlab en el server propio

Perfecto. Confirme si los siguientes parámetros son correctos:

- Autenticación: OAuth2
- Gestión de secretos: Variables de entorno
- Infraestructura: Servidor propio
- CI/CD: GitHub Actions o GitLab

¿Desea ajustar algo antes de continuar?

¿Confirmás la especificación? (enter = continuar, o escribí qué ajustar)

>

Auditoría de coherencia – semáforo: ● amarillo

● [gestion-secretos-insegura] Uso de variables de entorno para gestión de secretos en un entorno de servidor propio, lo que expone riesgos de seguridad si el servidor no está correctamente protegido.

Corrección propuesta: Implementar un sistema de gestión de secretos más seguro, como un servicio de almacenamiento de secretos (ej: AWS Secrets Manager, HashiCorp Vault) o usar herramientas de encriptación para variables de entorno en el servidor.

¿Asumís el riesgo 'gestion-secretos-insegura'? (s = asumir / N = corregir)

> S

Riesgo asumido y documentado: gestion-secretos-insegura

Auditoría de coherencia – semáforo: ● verde

Sin hallazgos: la especificación es coherente.

¿Cómo querés nombrar la carpeta del proyecto? (opcional – se genera en un directorio propio de tu sesión en el servidor, no en tu computadora; lo descargás como .zip al final)

> beta_datos_transformer

Proyecto generado con la plantilla 'fastapi' en

/tmp/pcia-web-sesiones/ef47cbf77526434f9dfcbdb7b8200ce9c/beta_datos_transformer (15 archivos):

- .env.example
- .github/workflows/ci.yml
- .gitignore

- Dockerfile
- README.md
- compose.yaml
- docs/adr/ADR-001-decisiones-iniciales.md
- pyproject.toml
- src/transformar_data/__init__.py
- src/transformar_data/config.py
- src/transformar_data/db.py
- src/transformar_data/main.py
- tests/conftest.py
- tests/test_config.py
- tests/test_health.py

Verificación de sintaxis: 11 ok, 0 con errores, 4 sin verificador.

✓ Todos los archivos verificables son válidos.

Verificación profunda (builds, smoke tests y linters)...

Resultado de la verificación profunda:

- ✓ [docker-build] ok
- ✓ [smoke-import-app] ok
- ✓ [lint-ruff] ok

⚠ El proyecto se entrega con 1 riesgo(s) asumido(s):

- gestion-secretos-insegura: Uso de variables de entorno para gestión de secretos en un entorno de servidor propio, lo que expone riesgos de seguridad si el servidor no está correctamente protegido.

Estado final de la entrega: ⚠ aprobado con advertencias

Especificación y registro del proyecto guardados en

/tmp/pcia-web-sesiones/ef47cbf77526434f9dfcbd7b8200ce9c/memory/transformar-data-20260731-173631.json
(duración total: 1288s)

Memoria actualizada: 1 proyecto(s) registrado(s).

Preferencias detectadas para precargar la próxima entrevista:

- lenguaje: python (en 1 de 1 proyectos)
- framework: fastapi, pandas (en 1 de 1 proyectos)
- arquitectura: en capas (en 1 de 1 proyectos)
- base_datos: postgresql (en 1 de 1 proyectos)
- autenticacion: oauth2 (en 1 de 1 proyectos)
- gestion_secretos: variables de entorno (en 1 de 1 proyectos)
- infraestructura: servidor propio (en 1 de 1 proyectos)
- ci_cd: github actions o gitlab (en 1 de 1 proyectos)